



TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE

This project is a **training site, not a real medical device** under any regulatory definition. The device described in this document — “SentinelFlow 500” — is entirely **fictional**, invented for this course: nothing here is a genuine IEC 62304 deliverable, a real regulatory submission, or evidence of any real organisation’s compliance with anything. It illustrates the *kind* of document a real SaMD/SiMD project would produce. See the [document register](#) for the full explanation.

This folder is a **second**, parallel document set to the one in [docs/](#). Where that set documents this training website’s own development — real evidence, but about a website, not a device — this one documents a fictional medical device, invented purely so a reader can see what IEC 62304 paperwork looks like when it’s actually about patient risk: hazards, essential performance, alarms, a hardware/software interface. None of it is real evidence of anything; all of it is illustrative.

What this is, and is not

SentinelFlow 500 does not exist. It is a fictional ambulatory infusion pump, invented for this course. Nothing in this folder is a real regulatory submission, a real device’s documentation, or evidence of any real organisation’s compliance with anything. See [01 — General Requirements & Classification](#) for the classification reasoning this whole set is built on.

The device

SentinelFlow 500 is a fictional ambulatory infusion pump for continuous and intermittent delivery of intravenous medication in a hospital setting. Clinical staff programme a flow rate, total volume, and duration; the pump delivers to those parameters until complete, paused, or stopped. The software under discussion throughout this document set is the pump’s embedded control software: it drives the delivery motor, reads the occlusion, air-in-line and door-open sensors,

validates programmed parameters against configurable dose-rate limits before delivery starts, and drives the display and alarms. Every document in this set describes the same device — this section is the one place its description lives, so it is referenced rather than repeated.

Classification: Class C (see [01](#) for the reasoning) — the control software is the sole control measure between a programming or delivery-monitoring failure and patient harm, which is exactly the condition IEC 62304 §4.3 names for Class C. Every one of the 13 process areas therefore applies in full, the same as the site's own [self-referential set](#) — though for this set, C is a genuine finding, not a chosen teaching target.

Document shape

Every document in this set is written as a **Standard Operating Procedure** — the shape a real QA/RA-controlled document takes, not a teaching walkthrough — so a reader sees what the *paperwork itself* looks like, not just an explanation of it. Each one follows the same nine sections:

1. **Purpose** — why this SOP exists, in one or two sentences.
2. **Scope** — what it covers and does not cover, and which safety classes it applies to.
3. **Responsibilities** — which role does what (Software Development Lead, QA/RA, Risk Manager, Configuration Manager — roles, not named individuals, since SentinelFlow 500 has no real staff).
4. **Definitions & Abbreviations** — only the terms this document actually uses.
5. **References** — the standard, related standards, and the other SOPs in this set it depends on or hands off to.
6. **What the standard requires** — the sub-clause requirements table, quoted from `applicability.json`, identical in content to the same table in the [self-referential set](#) — the requirements don't change with the device, only the answers do.
7. **Procedure** — the actual device-specific content: how this requirement is met for SentinelFlow 500. This is the section that varies most in completeness

between documents — see the register below.

- 8. **Records** — what evidence this SOP's execution produces and where it is retained.
- 9. **Revision History** — illustrative only; SentinelFlow 500 has no real revision history.

A document whose **Procedure** section is filled in is marked **SOP — worked example** in the table below. A document with sections 1–6 and 8–9 complete, but Procedure still a placeholder, is marked **SOP — template**: the shape a real document would have, with the device-specific detail not yet written. Neither is a bare skeleton — the requirements table, purpose, scope, responsibilities and references are real content either way.

Why this set exists alongside the other one

The [self-referential set](#) is genuinely useful — everything in it is true, which a fabricated device's documentation cannot claim to be — but it structurally cannot touch anything that depends on the software actually being part of a medical device: hazardous situations, essential performance, alarm conditions, hardware/software interfaces. This set exists to cover exactly that gap. Read together, the two sets show a learner both things: genuine traceability discipline (the reflexive set) and what device-specific content actually looks like (this one).

The document set

Same shape as the other set — one file per process area, in clause order. Every document now exists as a properly-shaped SOP (see "Document shape" above) — none are bare skeletons — but they differ in how much of the **Procedure** section, the device-specific content, has actually been written. Status reflects that, not what's planned to exist eventually.

#	DOCUMENT	CLAUSE	STATUS
01	General Requirements & Classification	Clause 4	SOP — worked example

#	DOCUMENT	CLAUSE	STATUS
02	Software Development Plan	Clause 5.1	SOP — template
03	Software Requirements Specification	Clause 5.2	SOP — template
04	Software Architecture	Clause 5.3	SOP — template
05	Software Detailed Design	Clause 5.4	SOP — template
06	Unit Implementation & Verification	Clause 5.5	SOP — template
07	Software Integration & Testing	Clause 5.6	SOP — template
08	Software System Testing	Clause 5.7	SOP — template
09	Software Release	Clause 5.8	SOP — template
10	Software Maintenance Plan	Clause 6	SOP — template
11	Software Risk Management File	Clause 7	SOP — partial worked example
12	Software Configuration Management Plan	Clause 8	SOP — template
13	Software Problem Resolution Process	Clause 9	SOP — template

Doc 11's hazard analysis (§7.1) is filled in — it carries the same three failure modes doc 01's classification rationale depends on — because a risk file with no hazards would leave doc 01's Class C finding unsupported; the rest of its Procedure (risk control verification, traceability) is still template, pending docs 03 and 04.

Reading these documents

Same online/PDF pattern as the other set — rendered by the main project's `docs/render.py` and `docs/render_pdf.py`, which both now cover this folder too. See the [Learn page](#): every clause card's Preview and Download buttons point here.

The real thing

This set is free, illustrative, and — for 11 of 13 documents — a template rather than a finished example. If you want the real, editable SOP/QMS template packs this shape is modelled on, St John Lynch & Co publish them at stjohnlynch.com/toolkit, including a free "Software & AleMD SOPs and Templates" pack.